

APPENDIX A

Troubleshooting

Problems met while building this project, and how they were fixed.

Symptom	Cause	Fix
NameError: name 'pytest' is not defined (in a notebook)	pytest is a terminal command, not Python	Use <code>!{sys.executable} -m pytest</code> , or run it in a terminal
InconsistentVersionWarning: unpickle estimator from version 1.7.1 when using 1.7.2	Notebook used the system Python kernel, not the venv	Select kernel 'Python (food-waste venv)' and retrain
Port 8501 shows the IMDB app	Old Streamlit process still running	Ctrl+C in its terminal, or run the other app on <code>--server.port 8502</code>
docker compose: ports are not available ... bind	Port 8000 or 8501 already in use on the host	Stop local uvicorn / streamlit, then docker compose up
Docker: cannot find dockerDesktopLinuxEngine	Docker Desktop is not started	Start Docker Desktop and wait until it is running
/health returns 503	No model file yet	<code>python -m foodwaste.train</code>
UI says 'local model (API offline)'	API not running or wrong API_URL	Start uvicorn or docker compose
Opened the wrong project folder in Jupyter	Several similar folders under MLOPS	Use MLOPS/food-waste-mlops/notebooks/walkthrough.ipynb
A server cell freezes the notebook	uvicorn / streamlit run until stopped	Start servers in a terminal, not a notebook cell

APPENDIX B

Command cheat sheet

Everything you need, in the order you normally use it.

```
# 0. once per terminal
cd C:\Users\priya\MLOPS\food-waste-mlops
.\env\Scripts\Activate.ps1

# 1. train (with quality gate)
python -m foodwaste.train

# 2. test
pytest -v

# 3. serve
uvicorn api.main:app --reload          # API  http://localhost:8000/docs  (terminal 1)
streamlit run app/streamlit_app.py    # UI   http://localhost:8501      (terminal 2)

# 4. monitor
python -m foodwaste.drift --simulate heatwave
python -m foodwaste.drift --from-logs

# 5. containers
docker compose up --build
docker compose down

# 6. ship
git add . ; git commit -m "message" ; git push  # GitHub Actions runs the CI pipeline
```

APPENDIX C

Next improvements

- **Experiment tracking and model registry** with MLflow: log every run's parameters, metrics and model, and promote versions from Staging to Production.
- **Automated retraining**: a scheduled GitHub Action that runs the drift check on the prediction log and retrains when drift is found.
- **Cloud deployment**: push the image to a registry (Docker Hub, AWS ECR) and run it on a managed service (AWS App Runner / ECS, Azure Container Apps, Google Cloud Run).
- **Performance monitoring**: once the real waste level is known each evening, join it with the logged predictions and track live macro F1, not only input drift.
- **Data versioning** with DVC, so every model can be traced to an exact dataset version.
- **Hyperparameter tuning** (GridSearchCV or Optuna) inside the training step, still behind the quality gate.